

G0002a: Statistical Data Analysis: Project 1 - 2025/2026 Spring

Luisa Scaroni

```
# Load libraries here
```

```
library(car)
library(moments)
library(robustHD)
library(robustbase)
library(mrfDepth)
library(readxl)
library(psych)
library(corrplot)
library(plot3D)
library(MASS)
library(Hmisc)
library(moments)
library(rrcov)
library(factoextra)
library(dplyr)
```

Loading Data

```
set.seed(1085999)
sweetsdata=read.csv("sweets_data.csv")
X=as.matrix(sweetsdata[, -9])
y=factor(sweetsdata$labels, levels = c("biscuits", "cakes"))
mygroup_ind=which(rmultinom(1, 1, rep(1/2,2)) == 1)
mygroup=levels(y)[mygroup_ind]
mysample=sample(which(y == mygroup), 200)
mydatafull=data.frame(X[mysample, ])
mytrainingdata=mydatafull[1:125, ]
myvalidationdata=mydatafull[126:200, ]
```

Questions

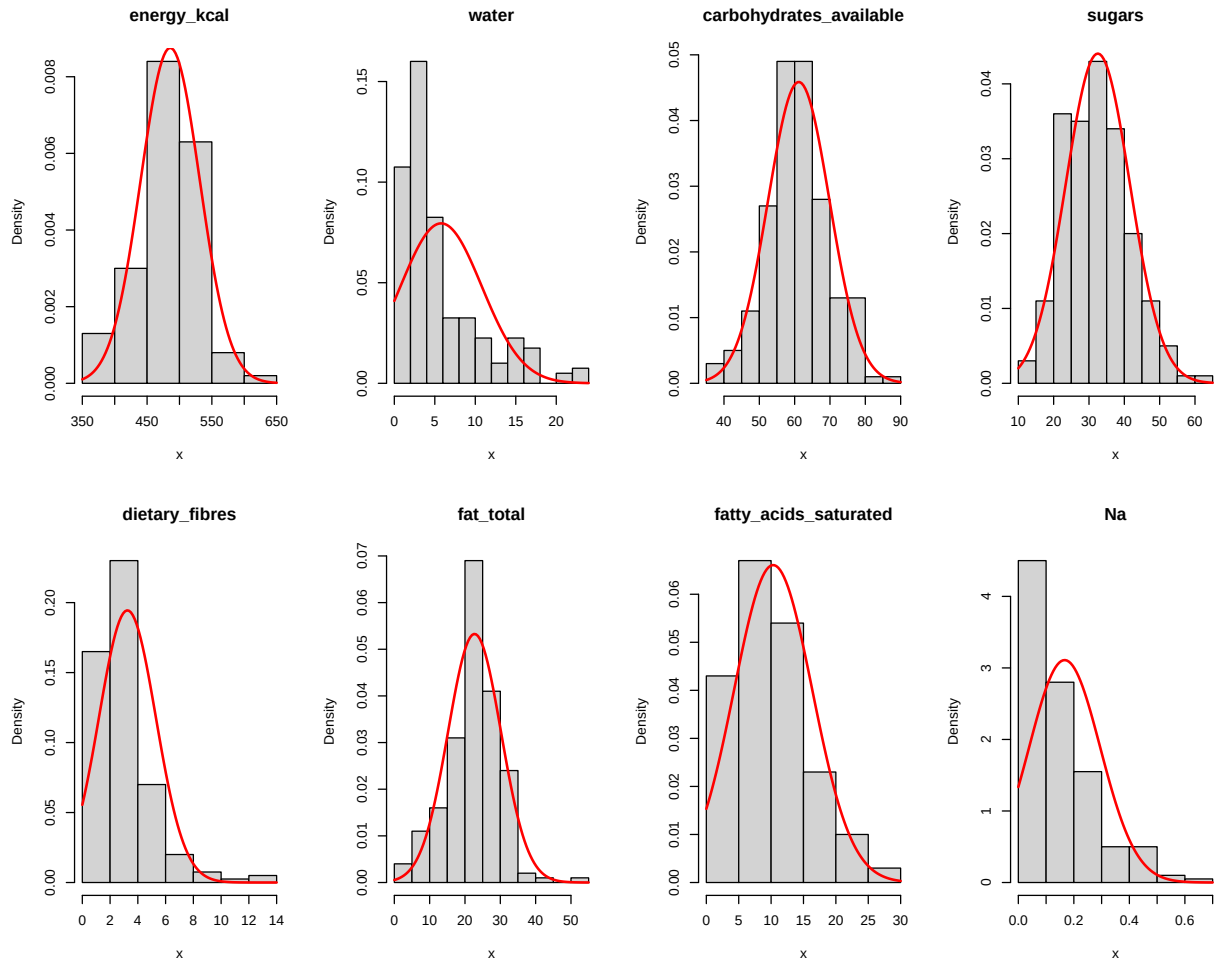
Question 1

To analyze the full 200 observations, we use the 'mydatafull'. We start with an exploratory analysis using histograms:

```
summary(mydatafull)
```

```
##   energy_kcal      water      carbohydrates_available      sugars
##   Min.   :365.0   Min.    : 0.500   Min.    :37.00      Min.    :13.00
##   1st Qu.:460.0   1st Qu.: 2.500   1st Qu.:56.00      1st Qu.:25.23
##   Median :490.0   Median : 3.600   Median :61.00      Median :32.00
##   Mean   :485.7   Mean    : 5.780   Mean    :61.17      Mean    :32.51
##   3rd Qu.:515.0   3rd Qu.: 7.625   3rd Qu.:66.00      3rd Qu.:39.00
##   Max.    :640.0   Max.    :24.000   Max.    :89.00      Max.    :61.00
##   dietary_fibres   fat_total   fatty_acids_saturated   Na
##   Min.    : 0.000   Min.    : 0.5   Min.    : 0.50      Min.    :0.0000
##   1st Qu.: 2.000   1st Qu.:19.0   1st Qu.: 6.00      1st Qu.:0.0800
##   Median : 2.500   Median :23.0   Median : 9.60      Median :0.1300
##   Mean    : 3.249   Mean    :22.7   Mean    :10.32      Mean    :0.1668
##   3rd Qu.: 3.550   3rd Qu.:27.0   3rd Qu.:14.00      3rd Qu.:0.2092
##   Max.    :14.000   Max.    :51.0   Max.    :30.00      Max.    :0.6900
```

```
par(mfrow=c(2,4))
for (i in 1:8){
  x=mydatafull[,i]
  hist(x, freq=FALSE, main=colnames(mydatafull)[i])
  mn=mean(x, na.rm=TRUE)
  std=sd(x, na.rm=TRUE)
  curve(dnorm(x,mean=mn,sd=std), col='red', lwd=2, add=TRUE)
}
```



```
skewness(mydatafull[,5])
```

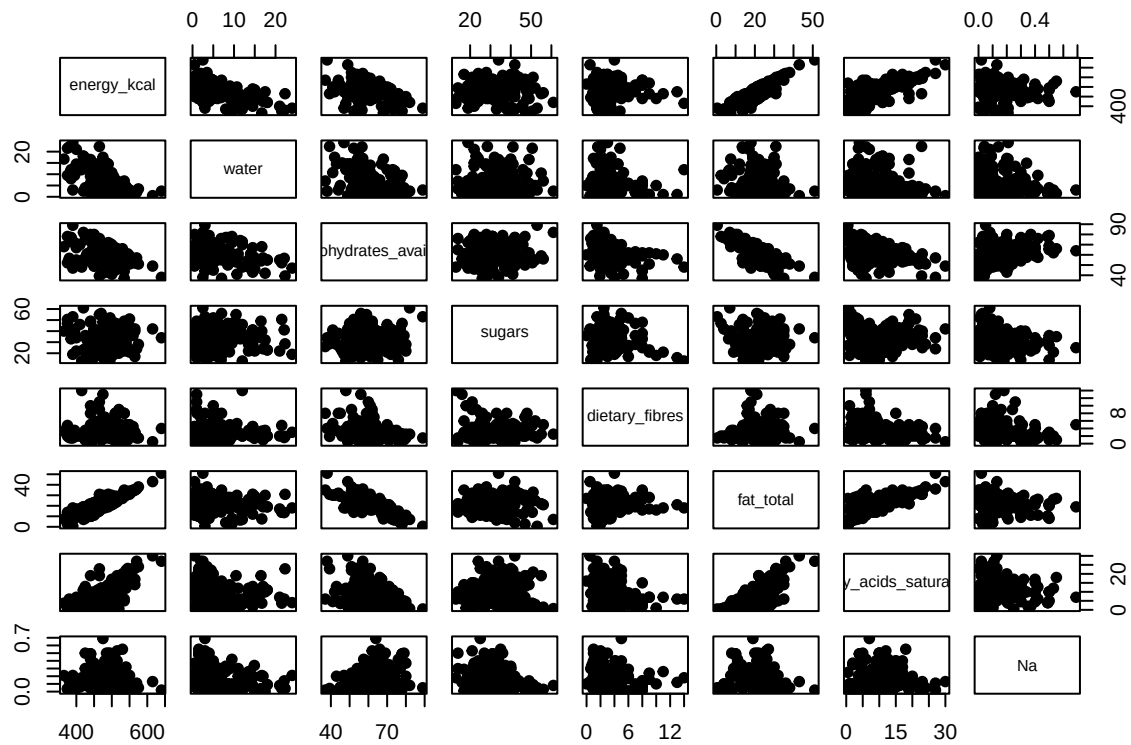
```
## [1] 2.231445
```

```
skewness(mydatafull[,1])
```

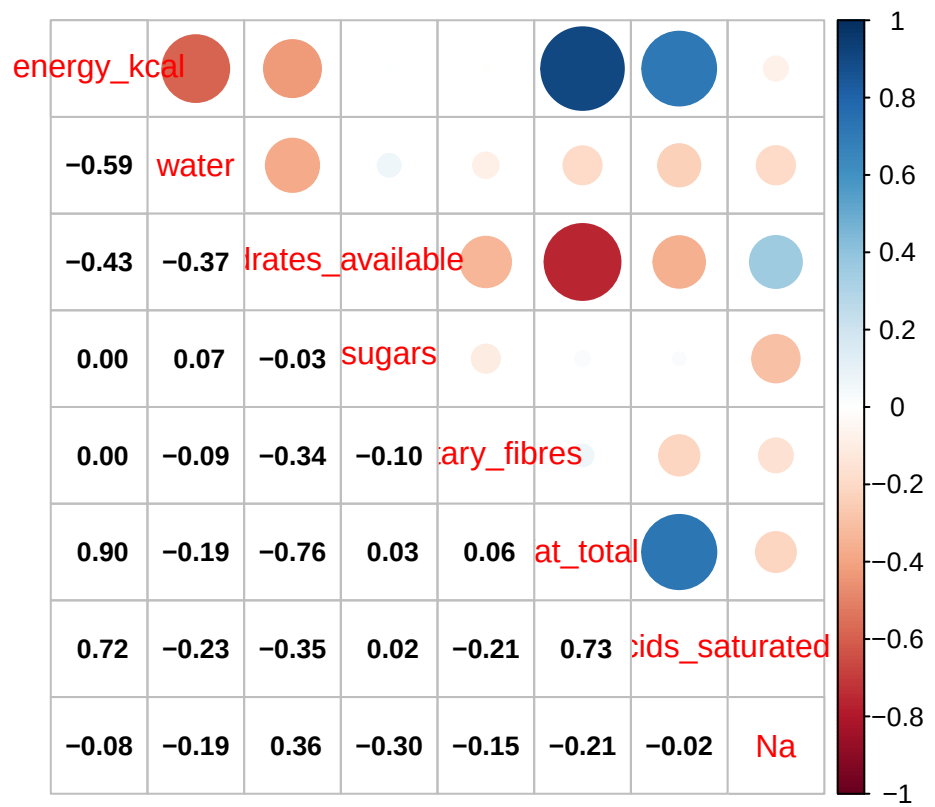
```
## [1] -0.1898936
```

We now plot the scatterplots and correlation plots:

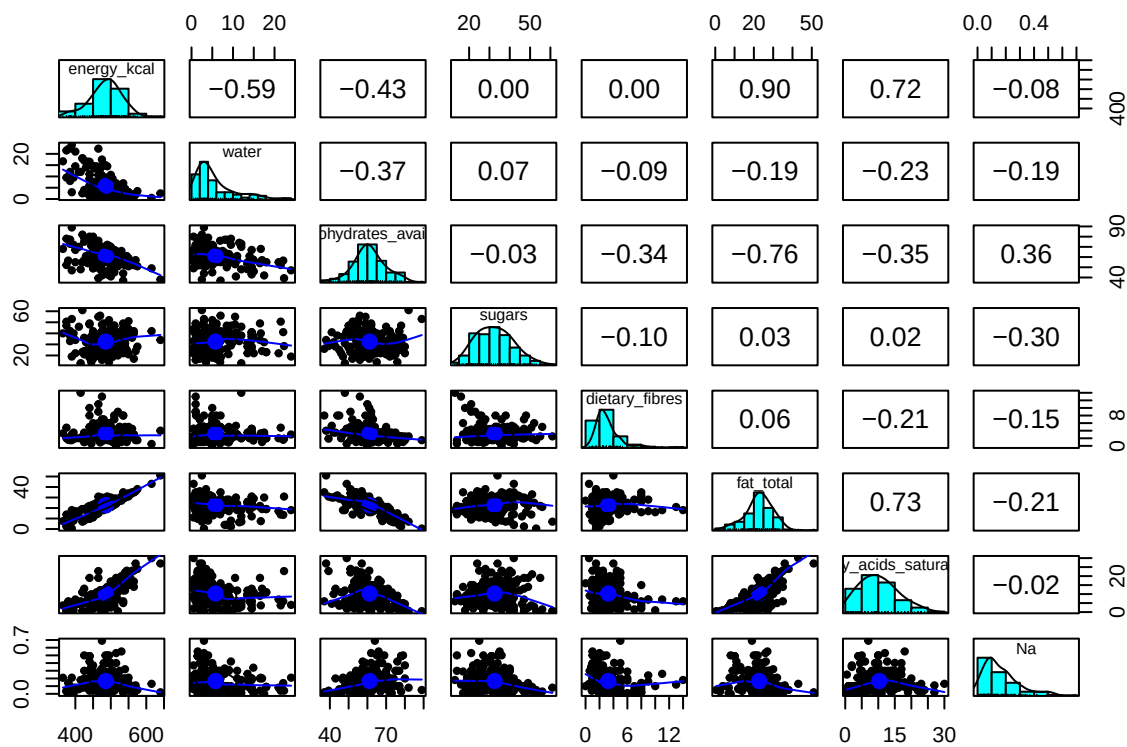
```
pairs(mydatafull, pch=19)
```



```
mydatafullcor=cor(mydatafull)
corrplot.mixed(mydatafullcor, number.cex=0.8, lower.col='black')
```

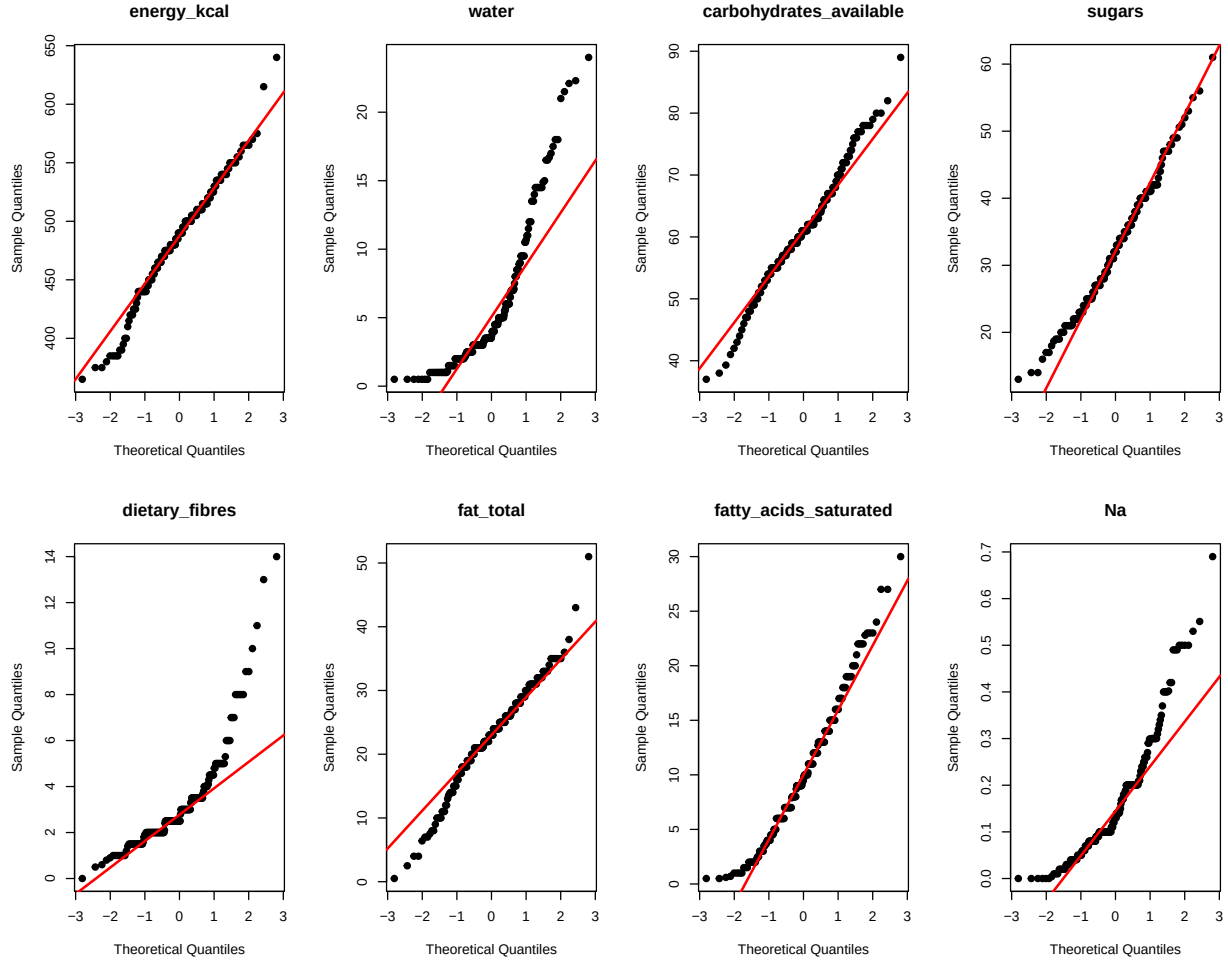


```
pairs.panels(mydatafull)
```



We now plot the QQ-plots with QQ-lines to study the normality more easily:

```
par(mfrow=c(2,4))
for(i in 1:8){
  qqnorm(mydatafull[,i],pch=19,main=colnames(mydatafull)[i])
  qqline(mydatafull[,i], col='red', lwd=2)
}
```



Discussion:

Through the histograms we can observe that some variables, such as “sugars”, “energy_kcal” and “carbohydrates_available”, have a roughly symmetric distribution. On the other hand variables like “dietary_fibres”, “water”, “Na” and “fatty_acids_saturated” show a right-skewed distribution, implying that their tails extend towards the higher values on the right.

Meanwhile studying the scatterplots we can see how the majority of the variables have zero or near-zero correlation. Indeed the points are widely scattered without a clear pattern, some examples of this are: “sugars” and “energy_kcal”, “dietary_fibres” and “energy_kcal”, “Na” and “acid_saturated”, “acid_saturated” and “sugars”, “Na” and “energy_kcal”, “sugars” and “fat_total”, “sugars” and “dietary_available”, “sugars” and “water”. Conversely, we can spot a high positive correlation between “fat_total” and “energy_kcal”, “fat_total_acid” and “saturate”, “acid_saturated” and “energy_kcal”. A very negative correlation characterizes “energy_kcal” and “water”, “carbohydrates_available” and “fat_total”. Then by using a correlation matrix (corrplot) we are able to check the exact values of the correlations and confirm our assumptions.

With the QQ-plots we can confirm the normal behavior for variables like “sugars” and “energy_kcal”, as the points closely follow the theoretical normal line. On the other hand we see a different behavior for “carbohydrates_available”, in fact while the central points align well there is a deviation at the extremes. Instead of concluding that this implies heavy tails, it makes more sense to assume that this suggests the presence of potential outliers at the edge of our sample. Meanwhile, the QQ-plots for the remaining variables deviate from the line, this confirms the non-normal and skewed distribution.

At the end, the majority of the histograms are right skewed (positive skewness). In these cases, the mean is larger than the median, moreover if we calculate the skewness we can find a value larger than 1. For example if we check “dietary_fibres” skewness we find a value of 2.23. On the contrary, for variables that look balanced like “energy_kcal”, the skewness is very close to zero (-0.189) confirming a symmetric shape.

Question 2

First we study the normality of “sugars” and “dietary_fibres”, then if we spot a non-normal behavior we apply a transformation.

```
sugar=mydatafull[,4]
fibres=mydatafull[,5]
```

Now that I have defined the two variables we can check if they are normal through different methods. We start with Shapiro-Wilk:

```
shapiro.test(sugar)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  sugar
## W = 0.98672, p-value = 0.0579
```

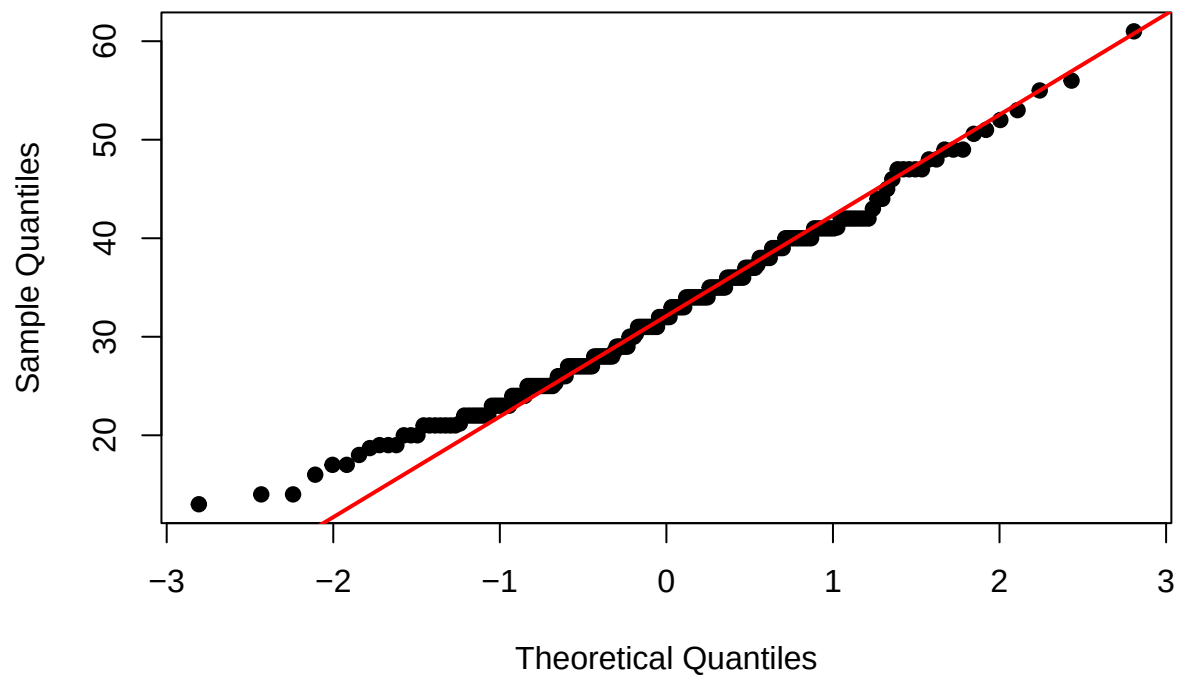
```
shapiro.test(fibres)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  fibres
## W = 0.79943, p-value = 2.648e-15
```

We plot again the QQ-norms to check the behaviors of the two to have another proof for this:

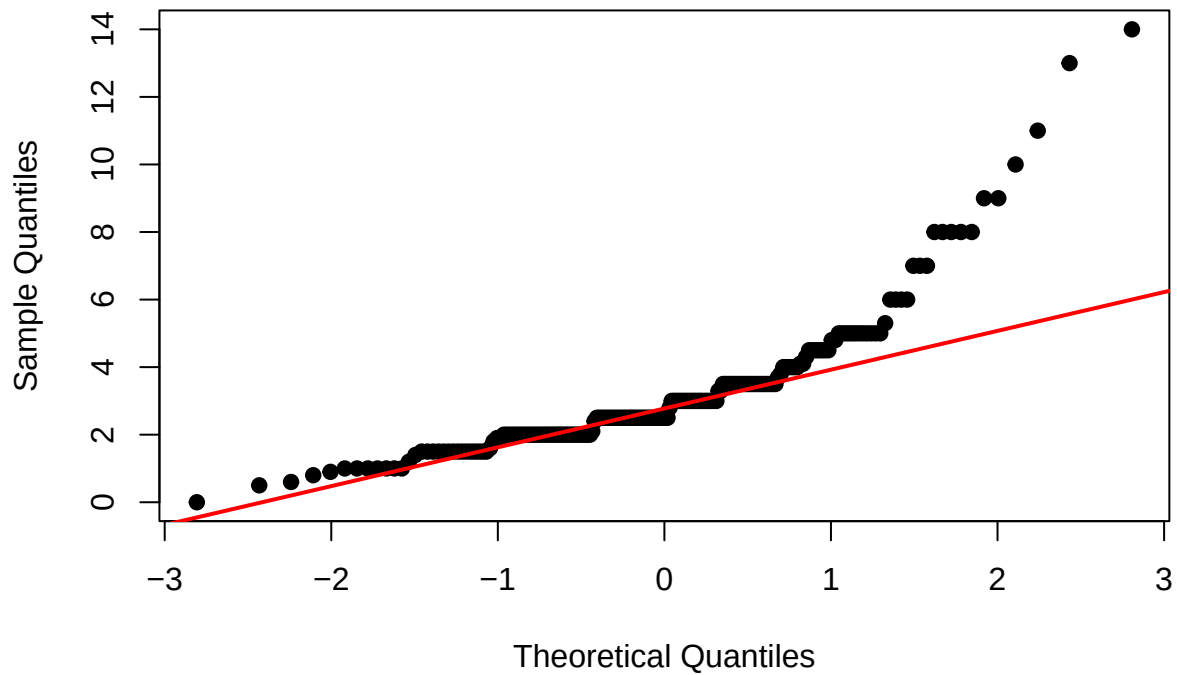
```
qqnorm(sugar, pch=19)
qqline(sugar, col='red', lwd=2)
```

Normal Q-Q Plot



```
qqnorm(fibres, pch=19)  
qqline(fibres, col='red', lwd=2)
```

Normal Q-Q Plot



```
skewness(sugar)
```

```
## [1] 0.3494163
```

```
skewness(fibres)
```

```
## [1] 2.231445
```

We try with some random transformations noting that some of them are not useful or efficient. We try with square root transformation:

```
transfiber1=sqrt(fibres)  
skewness(transfiber1)
```

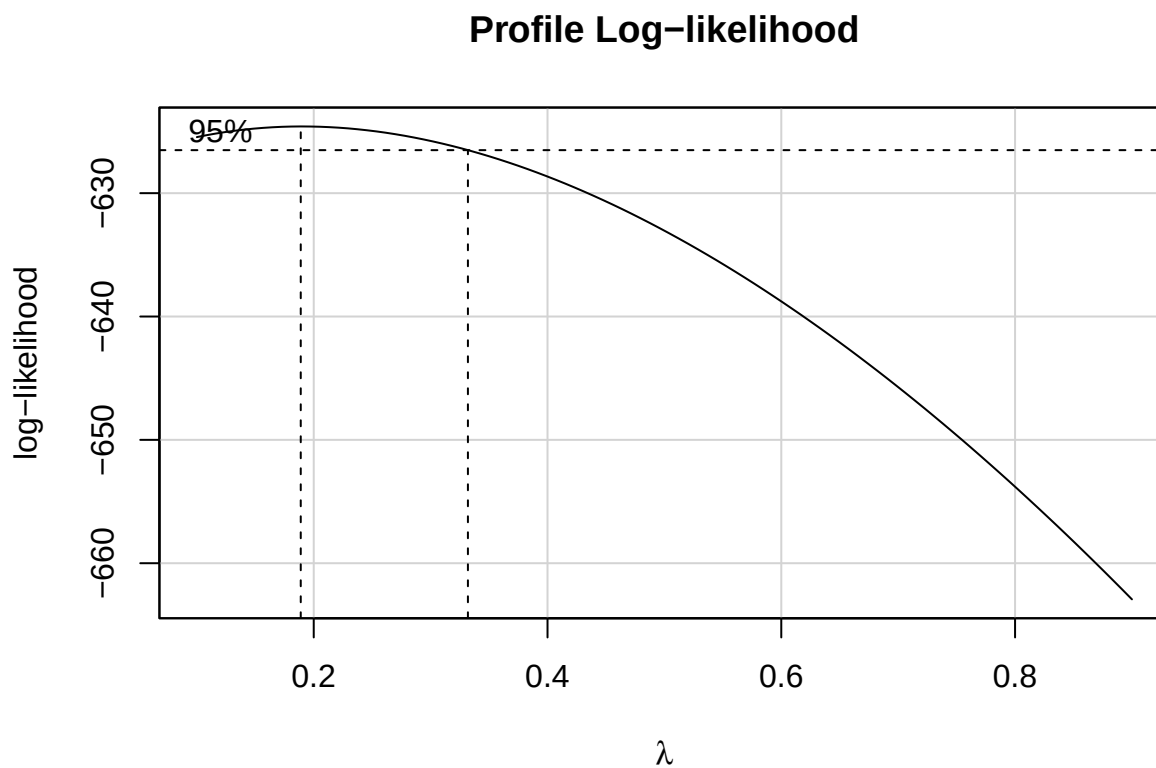
```
## [1] 0.9264142
```

```
shapiro.test(transfiber1)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data: transfiber1  
## W = 0.9292, p-value = 2.915e-08
```

We try with a Box-Cox transformation adding a constant because we have a value equal to zero:

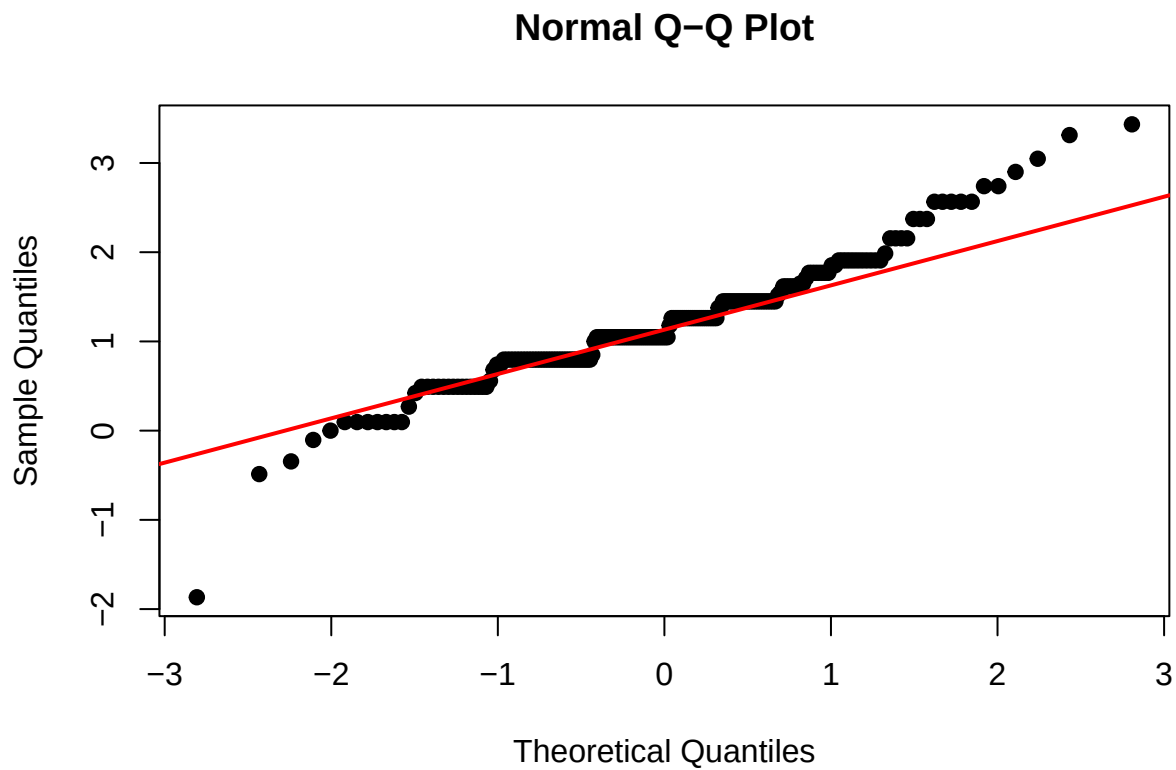
```
fibres1=fibres+0.1  
boxcoxfibres=boxCox(fibres1-1, seq(0.1, 0.9, 0.1))
```



```
lambda=boxcoxfibres$x[which.max(boxcoxfibres$y)]  
lambda
```

```
## [1] 0.1888889
```

```
transfibrescox=bcPower(fibres1,lambda)  
qqnorm(transfibrescox,pch=19)  
qqline(transfibrescox,col='red', lwd=2)
```



```
shapiro.test(transfibrescox)
```

```
##  
##  Shapiro-Wilk normality test  
##  
## data:  transfibrescox  
## W = 0.95619, p-value = 7.818e-06
```

```
skewness(transfibrescox)
```

```
## [1] 0.09341925
```

We now try with the inverse transformation:

```
invfibr=-1/(fibres+1)  
skewness(invfibr)
```

```
## [1] -1.727789
```

```
shapiro.test(invfibr)
```

```
##  
##  Shapiro-Wilk normality test
```

```
##  
## data:  invfibr  
## W = 0.89126, p-value = 7.226e-11
```

We try with a cube root transformation:

```
fibrcubic=fibres^(1/3)  
shapiro.test(fibrcubic)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  fibrcubic  
## W = 0.93407, p-value = 7.214e-08
```

```
skewness(fibrcubic)
```

```
## [1] 0.09432046
```

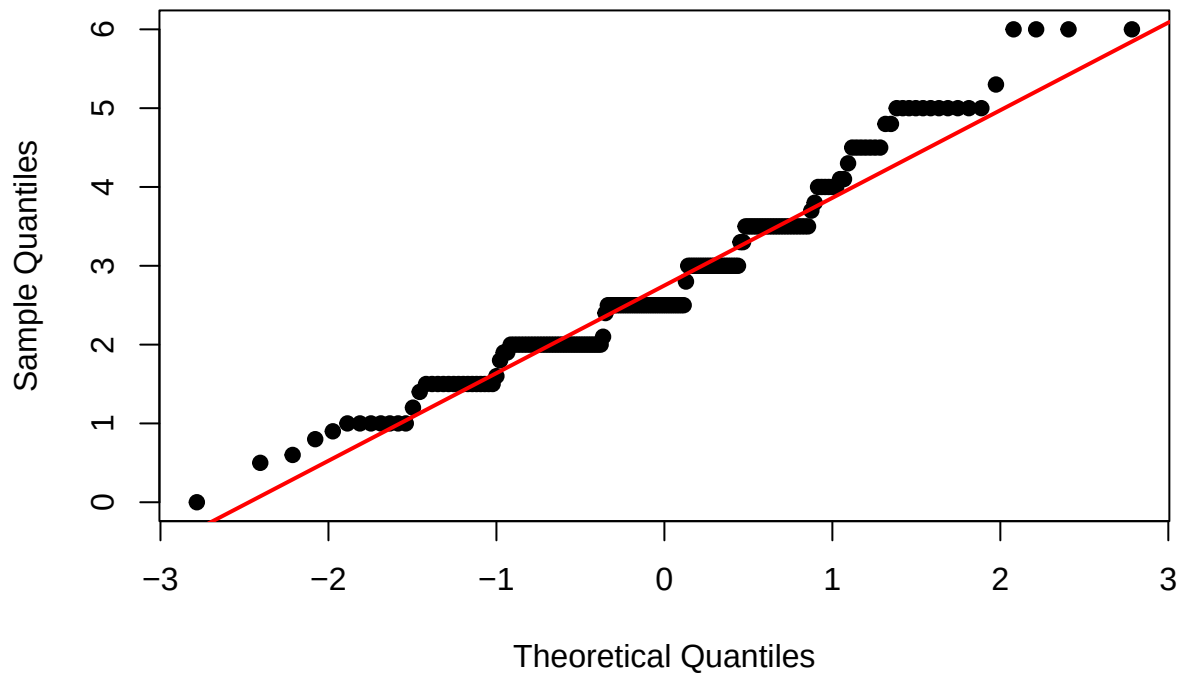
Now we try to eliminate the outliers to see if we are able to get a better behavior and check if they are the main cause for the non-normal behavior:

```
outliersfibres=boxplot.stats(fibres)$out  
fibresclean=fibres[!fibres %in% outliersfibres]  
shapiro.test(fibresclean)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  fibresclean  
## W = 0.9603, p-value = 4.196e-05
```

```
qqnorm(fibresclean, pch=19)  
qqline(fibresclean, col='red', lwd=2)
```

Normal Q-Q Plot

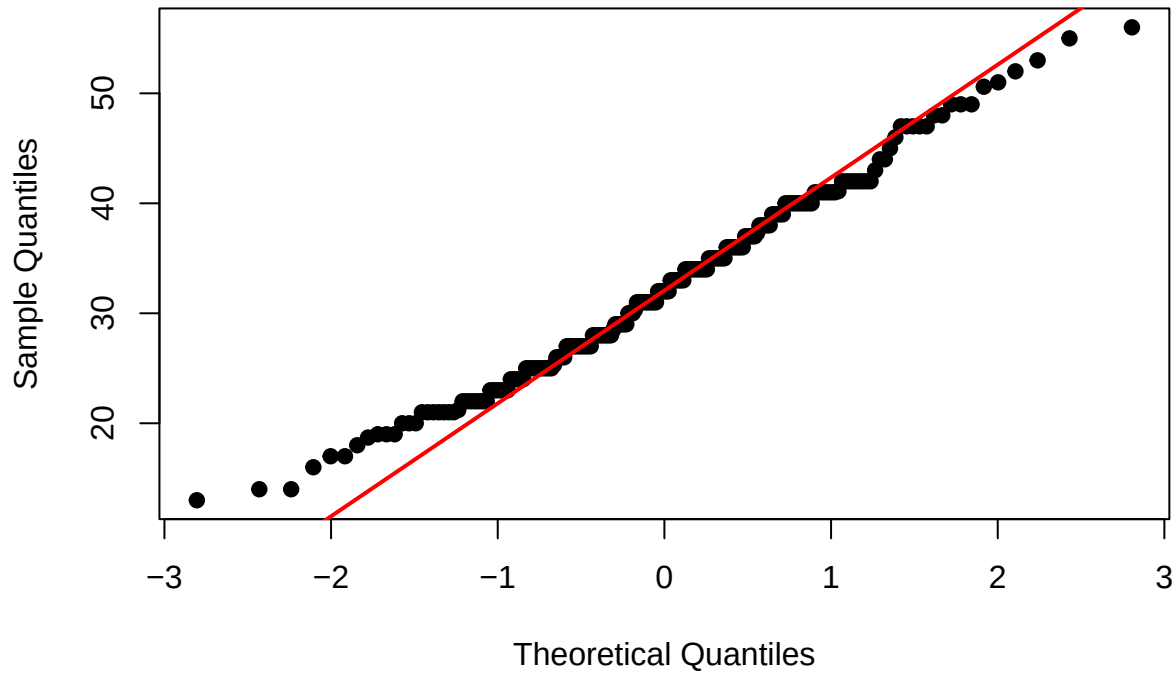


```
outlierssugar=boxplot.stats(sugar)$out
sugarclean=sugar[!sugar %in% outlierssugar]
shapiro.test(sugarclean)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  sugarclean
## W = 0.98761, p-value = 0.08034
```

```
qqnorm(sugarclean, pch=19)
qqline(sugarclean, col='red', lwd=2)
```

Normal Q-Q Plot



Discussion:

We start by calculating the p-value using the Shapiro-Wilk test. For the “sugars” variable, the p-value is greater (not a lot) than the 0.05 significance level, while for “dietary_fibres” it is close to zero (<0.05). So we can assume that “sugars” follows a normal distribution, which is also confirmed by the QQ-plot. On the other hand, “dietary_fibres” is not normally distributed, also the QQ-plot deviates a lot from the red theoretical line. Furthermore, “sugars” has a skewness close to zero, while “dietary_fibres” has a positive skewness greater than 1 (2.23), confirming what we have assumed in Question 1: heavily right-skewed distribution.

At this point we can try with some transformations to improve the normality of the variable “dietary_fibres”. We tested standard transformations such as the square root, the cube root and the inverse. The square root and the cube root yielded slightly better results compared to the original one, but still not normal. It is important to notice how we did not use the logarithmic transformation because in this specific case it would have made no sense, indeed “dietary_fibres” is measured in grams and contains values equal to zero. While we could have added a small constant to the variable and then apply the logarithmic transformation, the results would not have provided a better normalization than the Box-Cox transformation.

We conclude that the best transformation is the Box-Cox. Since “dietary_fibres” has some values equal to zero we added a small constant (0.1) to the variable before applying the transformation to make it computationally possible. We can notice how the Box-Cox transformed one does not pass the normality test with Shapiro-Wilk, but it provides a significant improvement in the QQ-plot, in the skewness and Shapiro-Wilk’s p-value values.

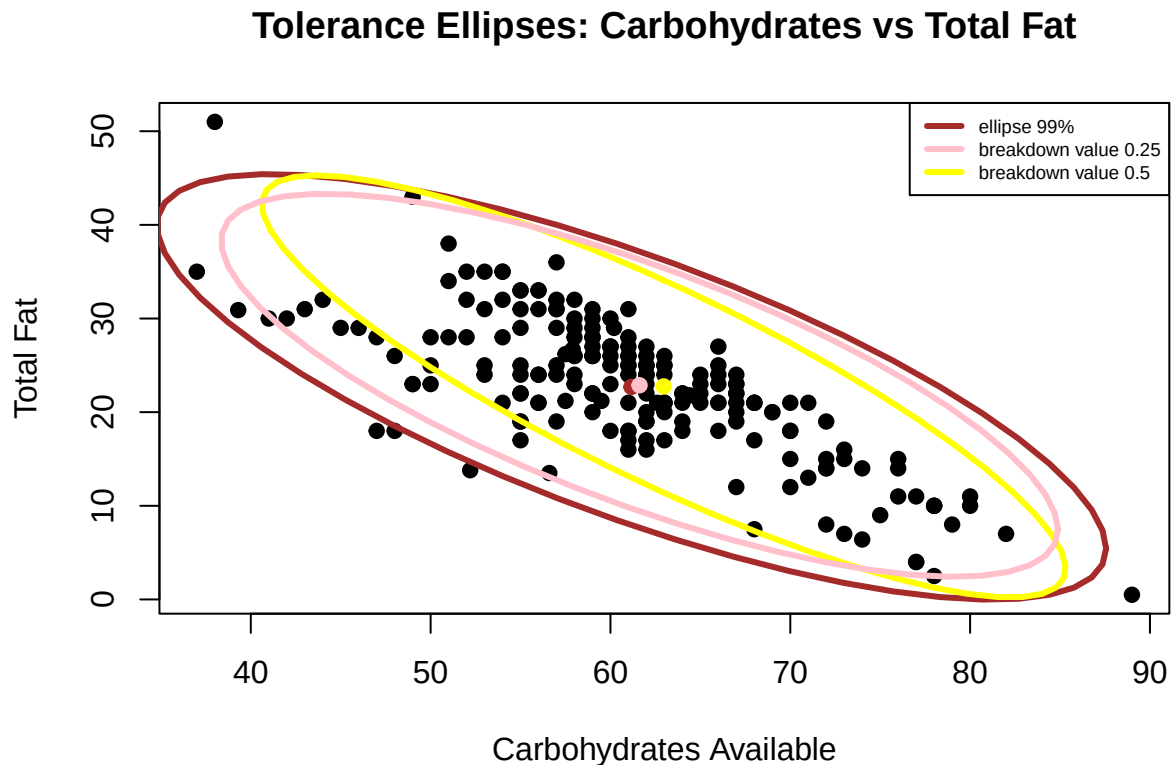
Finally, as an additional check we try to remove the outliers to study what kind of effect they have on the variable, we can notice how the Shapiro-Wilk p-value that we found is the highest one (not bigger than 0.05 but still better than before), so we can prove that the outliers affect the normality. We also tried to

remove the outliers for “sugars” although it was already normal and we can notice also in this case that the Shapiro-Wilk’s value improves.

Question 3

We have plotted the scatterplot and the classical tolerance ellipse. Next, we add the MCD-based tolerance ellipses using breakdown values of 50% and 25%, displaying them in different colors.

```
fat=mydatafull[,6]
carbo=mydatafull[,3]
data2d=cbind(carbo,fat)
radius99=sqrt(qchisq(0.99, df = 2))
m=colMeans(data2d)
S=cov(data2d)
plot(carbo, fat, pch=19, xlab="Carbohydrates Available", ylab = "Total Fat", main="Tolerance Ellipses: (
car::ellipse(m, S, radius99, col="brown", center.cex=1, lwd=3)
mcd25=covMcd(data2d, alpha=1-0.25)
mcd50=covMcd(data2d, alpha=1-0.5)
car::ellipse(mcd50$center, mcd50$cov, radius99, col="yellow", center.cex=1, lwd=3)
car::ellipse(mcd25$center, mcd25$cov, radius99, col="pink", center.cex=1, lwd=3)
legend('topright', legend=c('ellipse 99%', 'breakdown value 0.25', 'breakdown value 0.5'), col=c('brown'
```

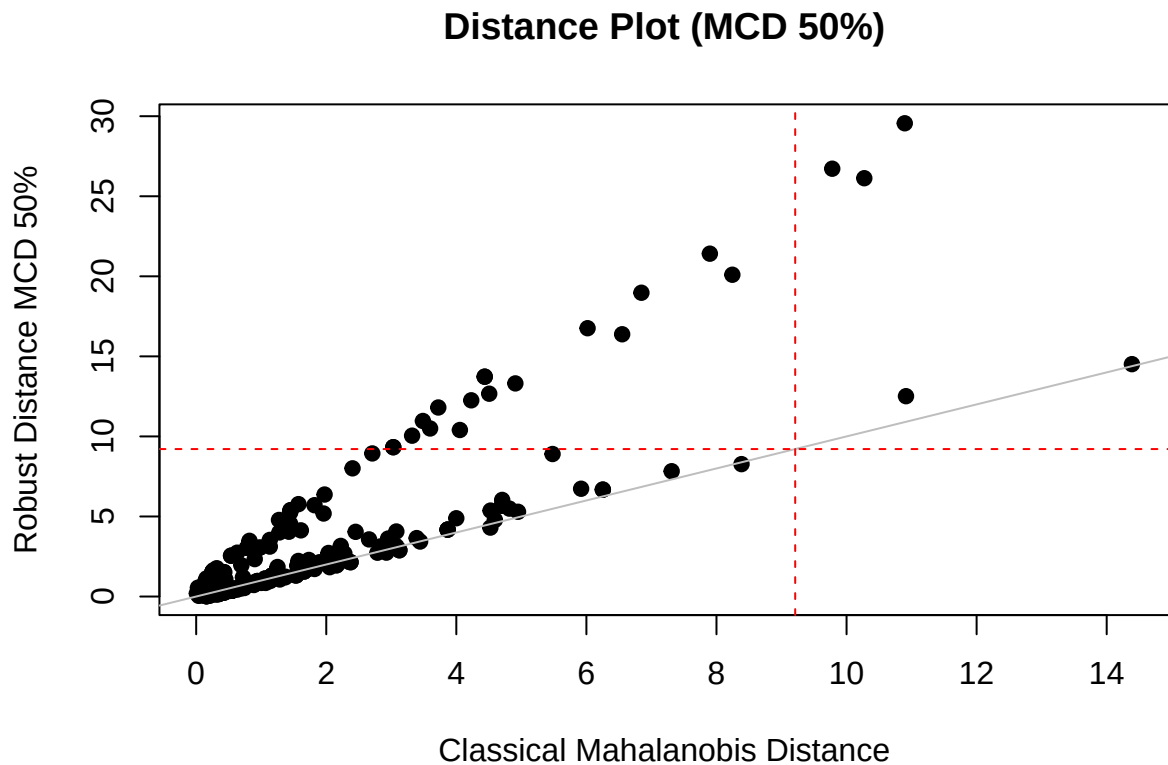


We then calculate the Mahalanobis distances to have a better view on everything:

```

distma=mahalanobis(data2d, m, S)
distmarobust50=mahalanobis(data2d, mcd50$center, mcd50$cov) #the most robust one among the two
distmarobust25=mahalanobis(data2d, mcd25$center, mcd25$cov )
#now we plot the three distances
plot(distma, distmarobust50, pch=19, ylab ="Robust Distance MCD 50%", xlab="Classical Mahalanobis Distance")
abline(a=0, b=1, col="gray")
abline(h=qchisq(0.99, 2), col="red", lty=2)
abline(v=qchisq(0.99, 2), col="red", lty=2)

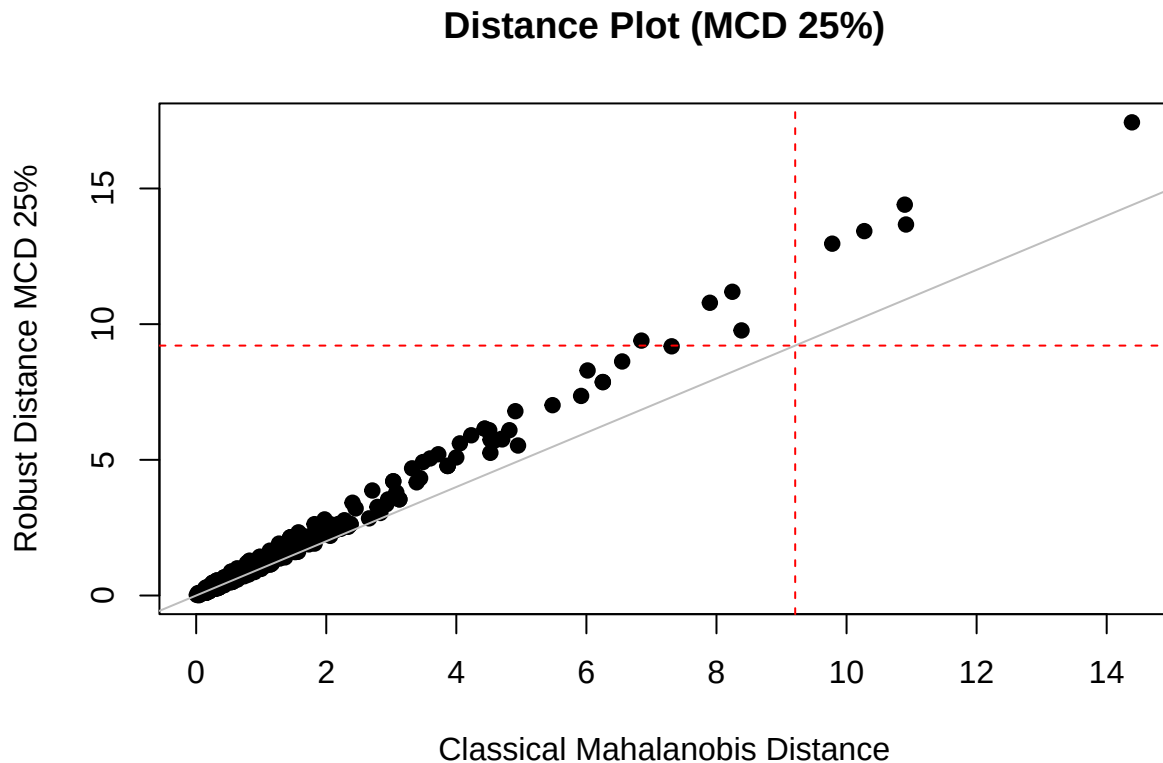
```



```

plot(distma, distmarobust25, pch=19, main="Distance Plot (MCD 25%)", xlab="Classical Mahalanobis Distance")
abline(a=0, b=1, col="gray")
abline(h=qchisq(0.99, 2), col="red", lty=2)
abline(v=qchisq(0.99, 2), col="red", lty=2)

```



Discussion:

We can observe the differences between the classical and the robust tolerance ellipse. The classical 99% tolerance is stretched towards the extreme values in the top-right area because the mean and the covariance matrix are sensitive to outliers. Meanwhile the MCD ellipses are tighter and capture data much better. It is easy to notice that the ellipse based on the 50% breakdown value is the most robust and compact, the 25% one is still good (not as much as 50%) but slightly influenced by extreme values.

Next, using the distances plot we are able to compare the Mahalanobis distances with the robust distances. First, the grey diagonal line ($x=y$) represents the point where the classical distance is equal to the robust one. We can see how the majority of the observation in the bottom-left part clusters around this line, showing that both the methods agree on their distance from the center, meanwhile the outliers deviate from it.

The red dashed lines are the 99% chi-square cutoff for outlier detection. For both of them in the top left part of the graph we can see a small classical distance with a large robust distance, so the classical estimator fails to detect outliers because the estimates are distorted by them.

Finally, while analyzing the differences between the two robust estimators, we can notice that the MCD50 estimator is more resistant. Indeed it is able to detect a larger number of masked outliers in the top-left quadrant and assigns them higher robust distances. On the opposite, the MCD25's estimates are partially dragged toward the outliers because of its lower breakdown value.

Question 4

We now have to perform a PCA analysis. ### Part a We need to check if we need to standardize the data, we start by studying the variances of the variables:

```
mydata1=mydatafull[,1:4]
mydata2=mydatafull[,6:8]
mydatafullnew=cbind(mydata1,transfibrescox , mydata2)
mytrainingdatanew=mydatafullnew[1:125, ]
myvalidationdatanew =mydatafullnew[126:200, ]
```

```
variances=apply(mytrainingdatanew, 2, var)
round(variances, 2)
```

```
##          energy_kcal          water carbohydrates_available
##          2319.52          24.82          67.89
##          sugars          transfibrescox          fat_total
##          81.28          0.46          59.19
##  fatty_acids_saturated          Na
##          37.59          0.02
```

```
summary(mytrainingdatanew)
```

```
##  energy_kcal          water          carbohydrates_available          sugars
##  Min.   :365.0    Min.   : 0.500    Min.   :38.00    Min.   :13.00
##  1st Qu.:460.0    1st Qu.: 2.500    1st Qu.:56.00    1st Qu.:25.30
##  Median :490.0    Median : 4.500    Median :60.00    Median :33.00
##  Mean   :484.8    Mean   : 6.046    Mean   :60.96    Mean   :32.51
##  3rd Qu.:515.0    3rd Qu.: 8.500    3rd Qu.:66.00    3rd Qu.:39.00
##  Max.   :640.0    Max.   :24.000    Max.   :80.00    Max.   :56.00
##  transfibrescox    fat_total    fatty_acids_saturated    Na
##  Min.   :-1.8672    Min.   : 2.5    Min.   : 0.50    Min.   :0.0000
##  1st Qu.: 0.7964    1st Qu.:18.0    1st Qu.: 6.00    1st Qu.:0.0800
##  Median : 1.0472    Median :24.0    Median : 9.10    Median :0.1000
##  Mean   : 1.2031    Mean   :22.7    Mean   :10.13    Mean   :0.1603
##  3rd Qu.: 1.6169    3rd Qu.:27.0    3rd Qu.:14.00    3rd Qu.:0.2010
##  Max.   : 3.4329    Max.   :51.0    Max.   :27.00    Max.   :0.6900
```

Discussion:

By calculating the variances of the training set we can see how it is necessary to standardize the data, indeed the various variances are on different scales. For example, “energy_kcal” has an extremely large variance (around 2000), whereas “Na” has a variance close to 0. Because the main aim of Principal Component Analysis is to maximize variance, using unscaled data would lead to wrong results. As a matter of fact, the variables with larger scales (like “energy_kcal”) would dominate on the smaller one implying that we would ignore the latter. Therefore, by standardizing we can ensure that all the variable contribute in the same way to the analysis.

Part b

```
trainpca=PcaClassic(mytrainingdatanew, scale=TRUE)
summary(trainpca)
```

```
##
## Call:
## PcaClassic(x = mytrainingdatanew, scale = TRUE)
## Importance of components:
##              PC1      PC2      PC3      PC4      PC5      PC6      PC7
## Standard deviation    1.8033 1.3545 1.0184 0.9977 0.75019 0.54545 0.14070
## Proportion of Variance 0.4065 0.2293 0.1296 0.1244 0.07035 0.03719 0.00247
## Cumulative Proportion 0.4065 0.6358 0.7655 0.8899 0.96024 0.99743 0.99991
##              PC8
## Standard deviation    0.02745
## Proportion of Variance 0.00009
## Cumulative Proportion 1.00000
```

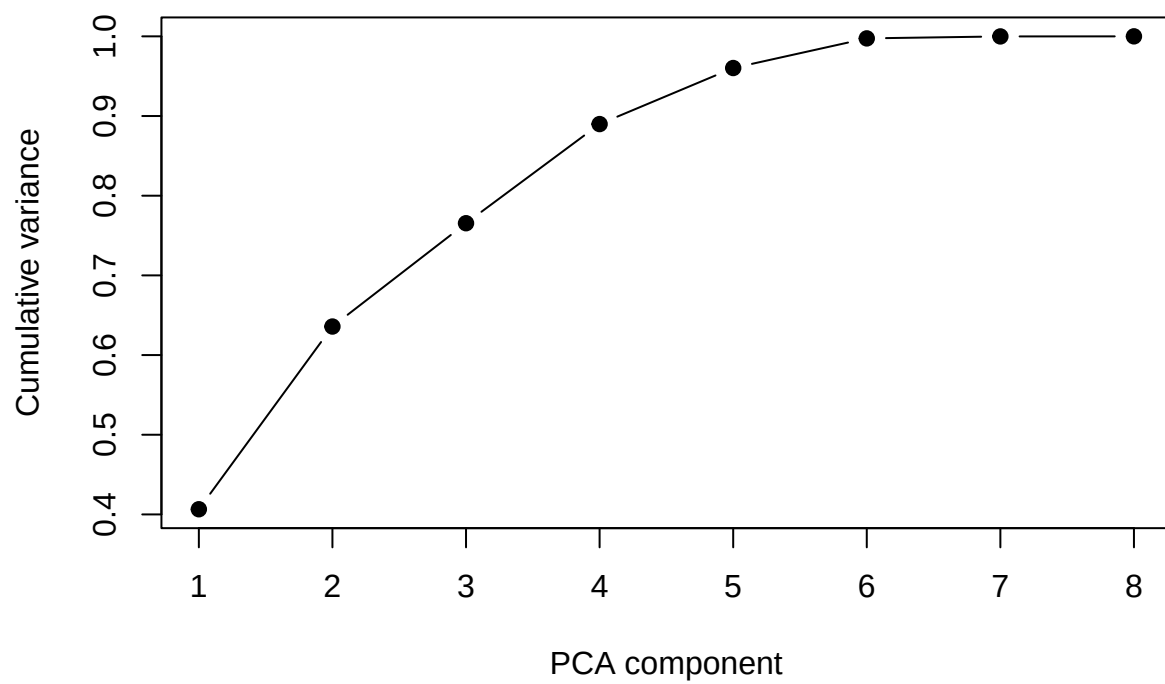
```
eigen=getEigenvalues(trainpca)
eigen
```

```
## [1] 3.2519748474 1.8345847793 1.0371142882 0.9954841537 0.5627801625
## [6] 0.2975122097 0.0197959488 0.0007536103
```

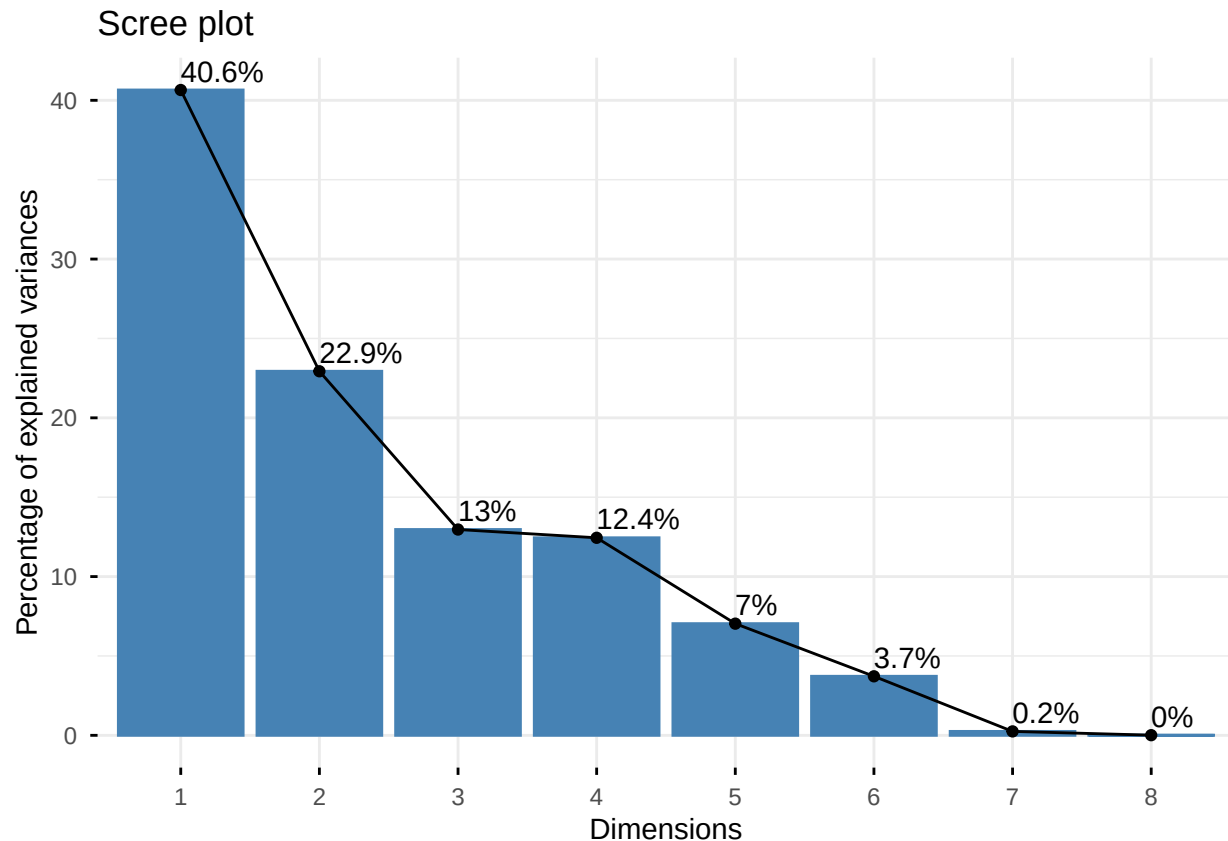
```
trainpca$loadings
```

```
##              PC1      PC2      PC3      PC4
## energy_kcal    0.52395098 -0.20053907 0.03050033 0.06362076
## water          -0.23894548 0.51711209 -0.06545727 -0.52005424
## carbohydrates_available -0.36856975 -0.44794166 0.28682868 0.23873256
## sugars          0.07608342 0.33183132 0.63117251 0.46279392
## transfibrescox 0.09191720 0.29863156 -0.61243026 0.62602605
## fat_total      0.54051539 0.04211551 -0.05880928 -0.13113219
## fatty_acids_saturated 0.45089170 -0.19492207 0.13885749 -0.21120155
## Na             -0.15112609 -0.50265920 -0.34106436 -0.02478269
##              PC5      PC6      PC7      PC8
## energy_kcal    -0.03715503 -0.293168716 0.2974563068 0.710262583
## water          0.25492570 0.169936117 0.4955679090 0.247600737
## carbohydrates_available -0.21122031 0.221203984 0.6512825277 -0.080786948
## sugars          0.52104399 -0.002518413 -0.0067330133 -0.001955968
## transfibrescox -0.01366027 0.311954663 0.1915576683 0.034559081
## fat_total      0.08399192 -0.245012428 0.4382927561 -0.652862112
## fatty_acids_saturated 0.08024554 0.821698363 -0.1136758425 0.016273335
## Na             0.77707846 -0.060838091 0.0009226298 0.001578610
```

```
plot(cumsum(trainpca$eigenvalues)/sum(trainpca$eigenvalues), type="b", pch=19,
main="", xlab="PCA component", ylab="Cumulative variance")
```



```
trainprcomp=getPrcomp(trainpca)
fviz_eig(trainprcomp, addlabels = TRUE)
```



We assume that we need only 4 Principal Components so we calculate the loadings of the four:

```
loadings4pc=getLoadings(trainpca)[,1:4]
print(loadings4pc)
```

##	PC1	PC2	PC3	PC4
## energy_kcal	0.52395098	-0.20053907	0.03050033	0.06362076
## water	-0.23894548	0.51711209	-0.06545727	-0.52005424
## carbohydrates_available	-0.36856975	-0.44794166	0.28682868	0.23873256
## sugars	0.07608342	0.33183132	0.63117251	0.46279392
## transfibrescox	0.09191720	0.29863156	-0.61243026	0.62602605
## fat_total	0.54051539	0.04211551	-0.05880928	-0.13113219
## fatty_acids_saturated	0.45089170	-0.19492207	0.13885749	-0.21120155
## Na	-0.15112609	-0.50265920	-0.34106436	-0.02478269

Discussion:

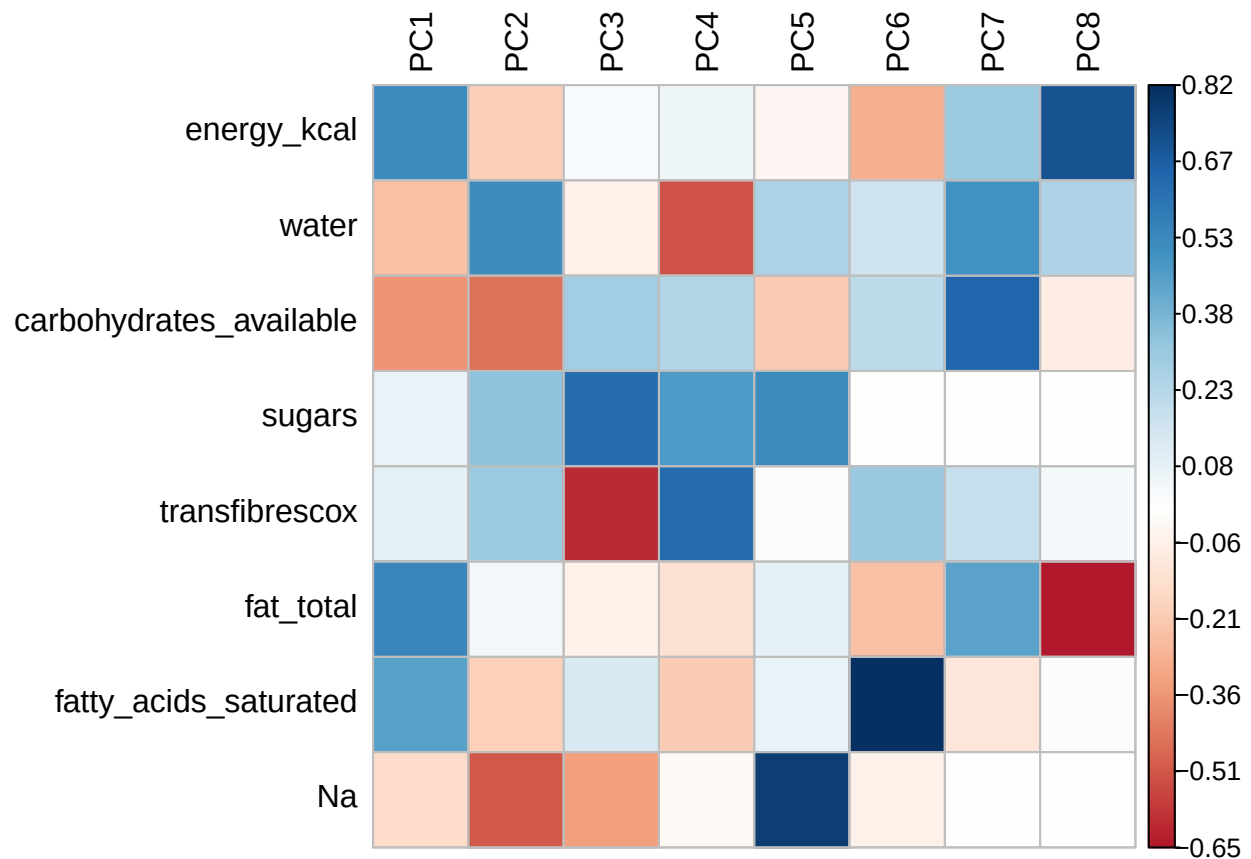
In order to understand how many principal components we need we applied two criteria: Kaiser-Guttman Rule and visual analysis of the variance plots (Scree Plot and Cumulative Variance Plot).

According to Kaiser-Guttman Rule we should select the components with eigenvalues greater than one. Only the first three components meet this criterion with eigenvalues of 3.25, 1.83 and 1.04. However, it is important to notice how the fourth component is a borderline case because its eigenvalue is 0.995 (very close to 1) and as we can see on the screeplot it has a very similar amount of variance compared to PC3 (13% and 12.4% respectively). The true part of the curve in which variance drops significantly and the plot flattens out occurs after the fourth component.

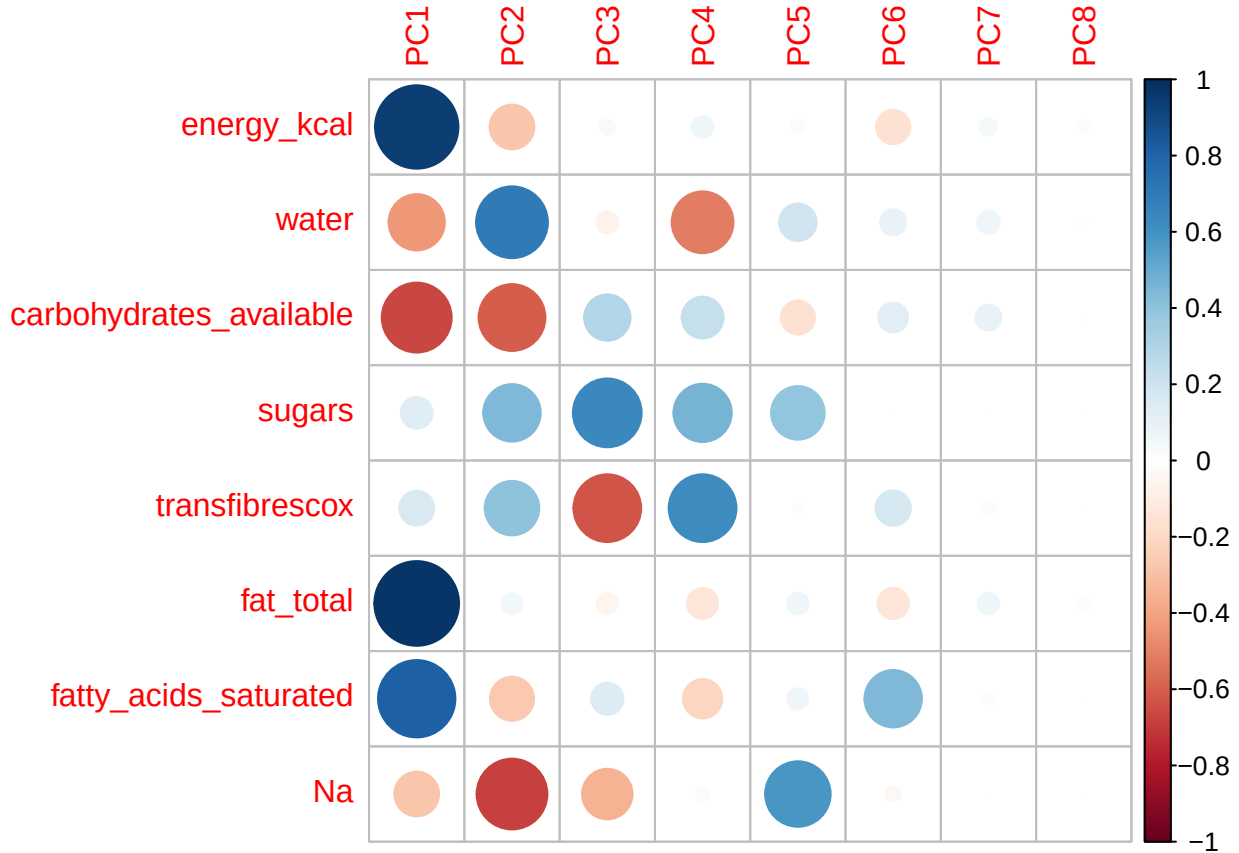
Finally, while the first three components capture the 76.55% of the total variance, including PC4 increases the cumulative explained variance to a better 88.99%. For these reasons, we have decided to select four components for the remaining analysis.

Part c

```
corrplot(trainpca$loadings, is.corr=FALSE, method='color', tl.col='black', addgrid.col = 'grey')
```



```
pcscores=getScores(trainpca)
correlation=cor(mytrainingdatanew, pcscores)
corrplot(correlation, method='circle')
```



Discussion:

We can now analyse the correlation matrix between the original variables and the Principal Components scores getting a nutritional interpretation of each PCs.

PC1 is strongly positively correlated with “fatty_acid_saturated”, “fat_total” and “energy_kcal”, while being negatively correlated with “carbohydrates_available”. This implies that PC1 probably captures the overall caloric and fat density of sweets, contrasting them with the products primarily composed of carbohydrates.

PC2 shows a strong positive correlation with “water” and a negative correlation with “carbohydrates_available” and “Na”. This suggest that this component distinguishes well-hydrated products from saltier and drier one.

PC3 displays a strong positive correlation with “sugars” but a negative one with “transfibres”. This indicates that the component differentiates fibrous product from very sweets ones.

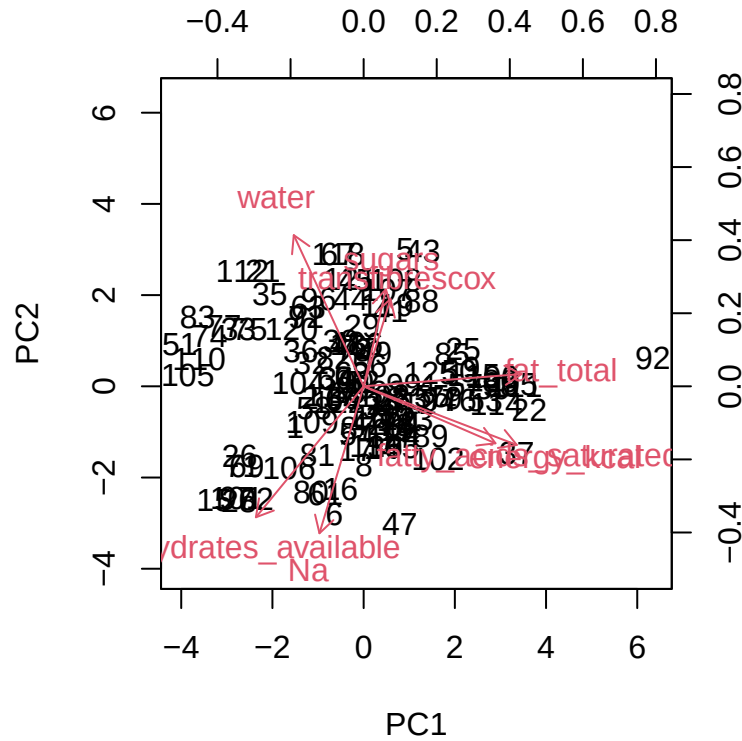
PC4 shows a strong positive correlation with “transfibrescox” and “sugars”, while having a strong negative one with “water”. This component seems to identify very dry but rich in fibres and sugars products.

It is important to explain the difference between PCA loadings (square plot) and the correlations (circle plots). Indeed if the loadings represent the weights defining the directions of the various PCs, the correlation plot show how much variance is captured by those components. As a consequence, after PC3 and PC4 the correlation circles fade to white. As a matter of fact, we selected only the first few components because the others contain zero variance and have no meaningful correlation with the variables.

Part d

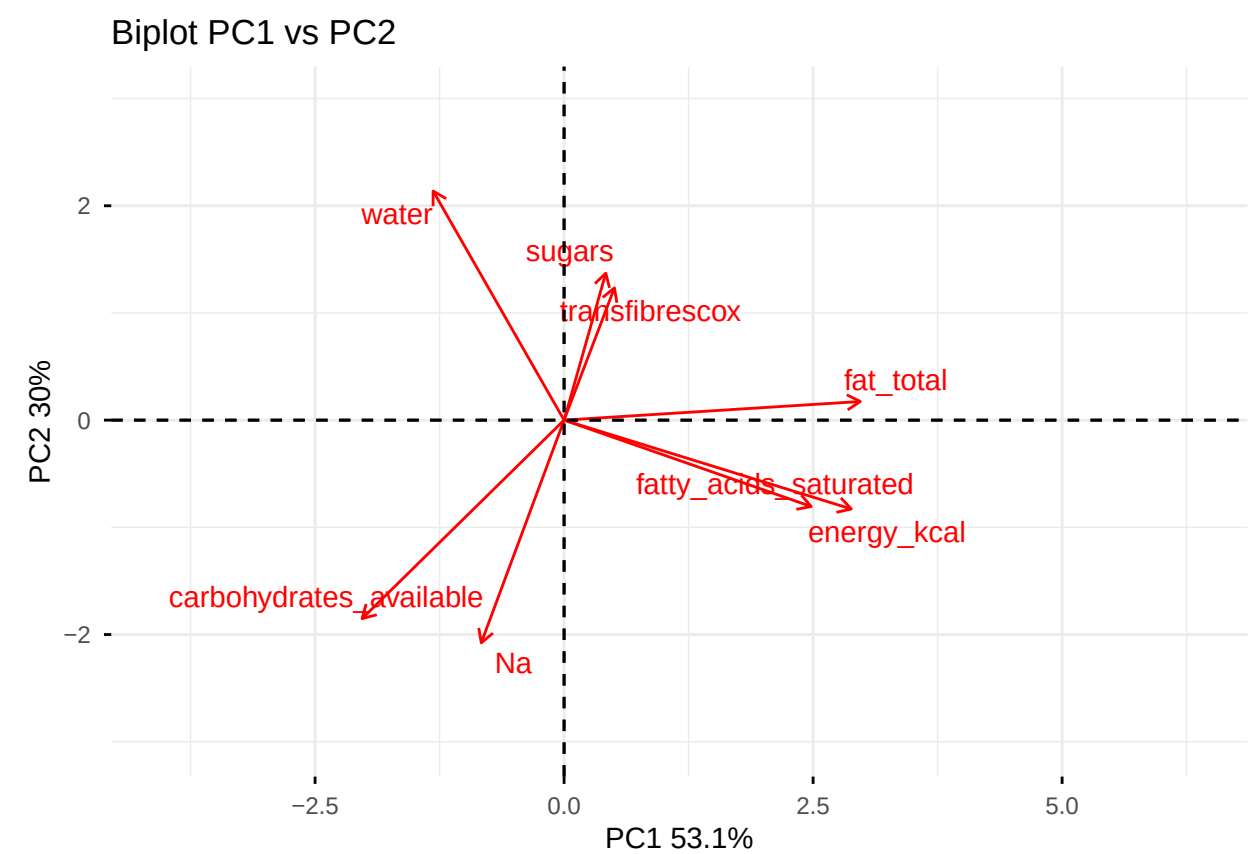
We now consider the biplots of the first three PCA:

```
trainpca3=PcaClassic(mytrainingdatanew, k=3, scale=TRUE, crit.pca.distances=0.99)
biplot(trainpca3, scale=0, cex=c(0,1) )
```

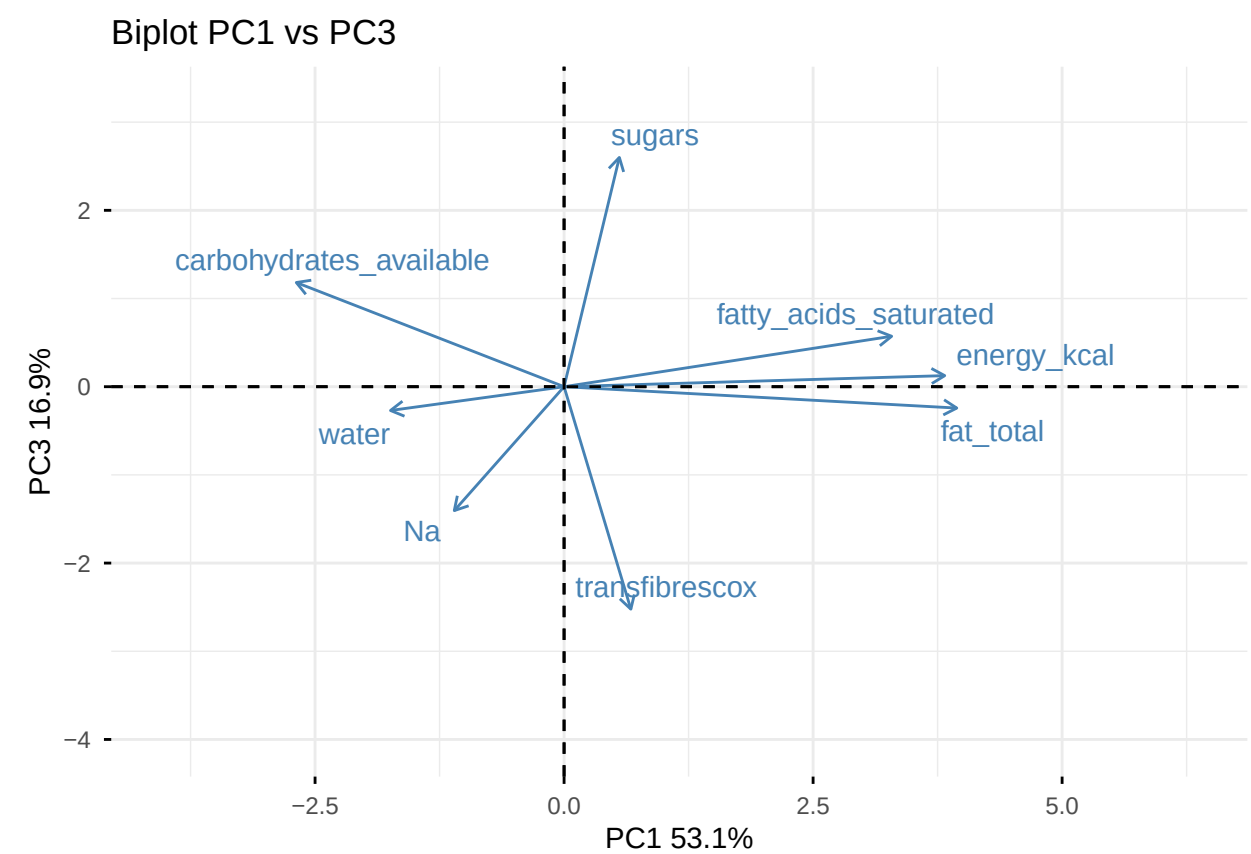


The previous one was too difficult to study so we use a different kind of plot:

```
trainprompt=getPrcomp(trainpca3)
fviz_pca_biplot(trainprompt, axes=c(1,2),col.var="red",repel=TRUE, title='Biplot PC1 vs PC2', xlab="PC1
```

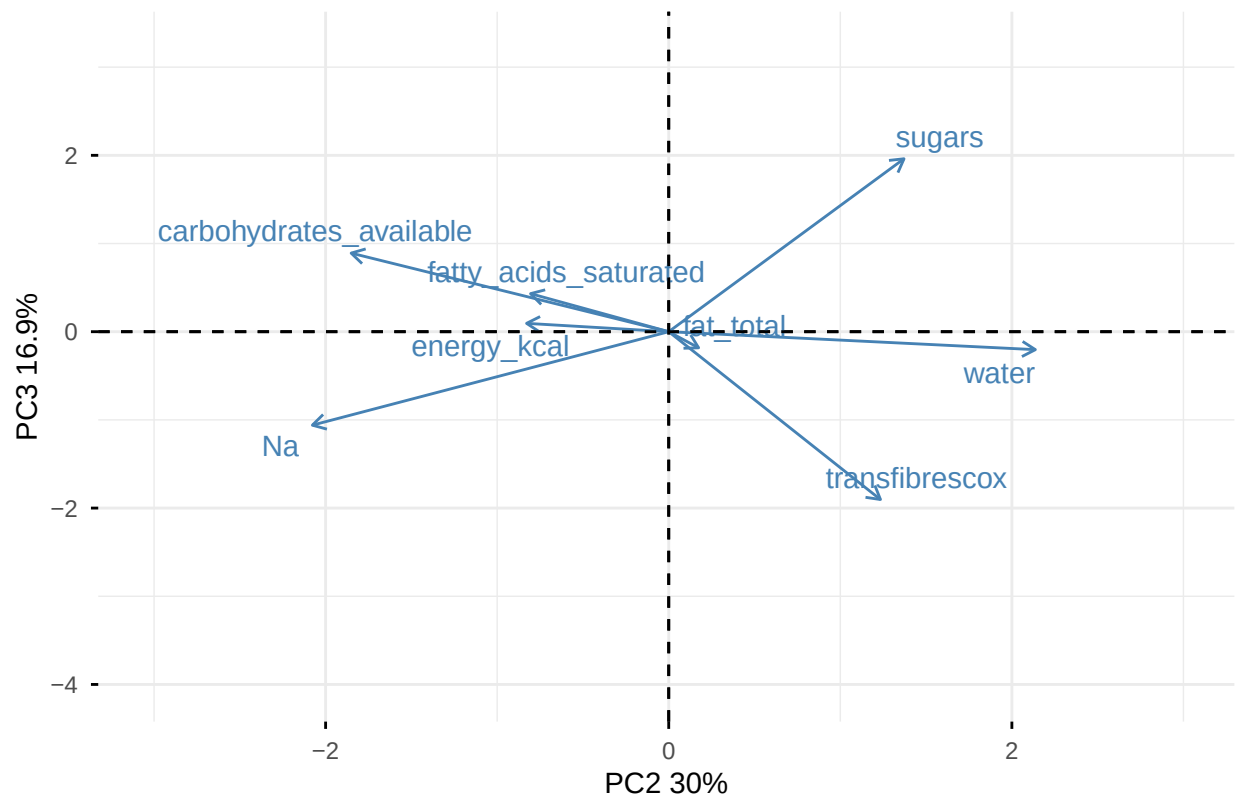


```
fviz_pca_biplot(trainprompt, axes=c(1,3), repel=TRUE, title='Biplot PC1 vs PC3', xlab="PC1 53.1%", ylab="PC3 30%")
```

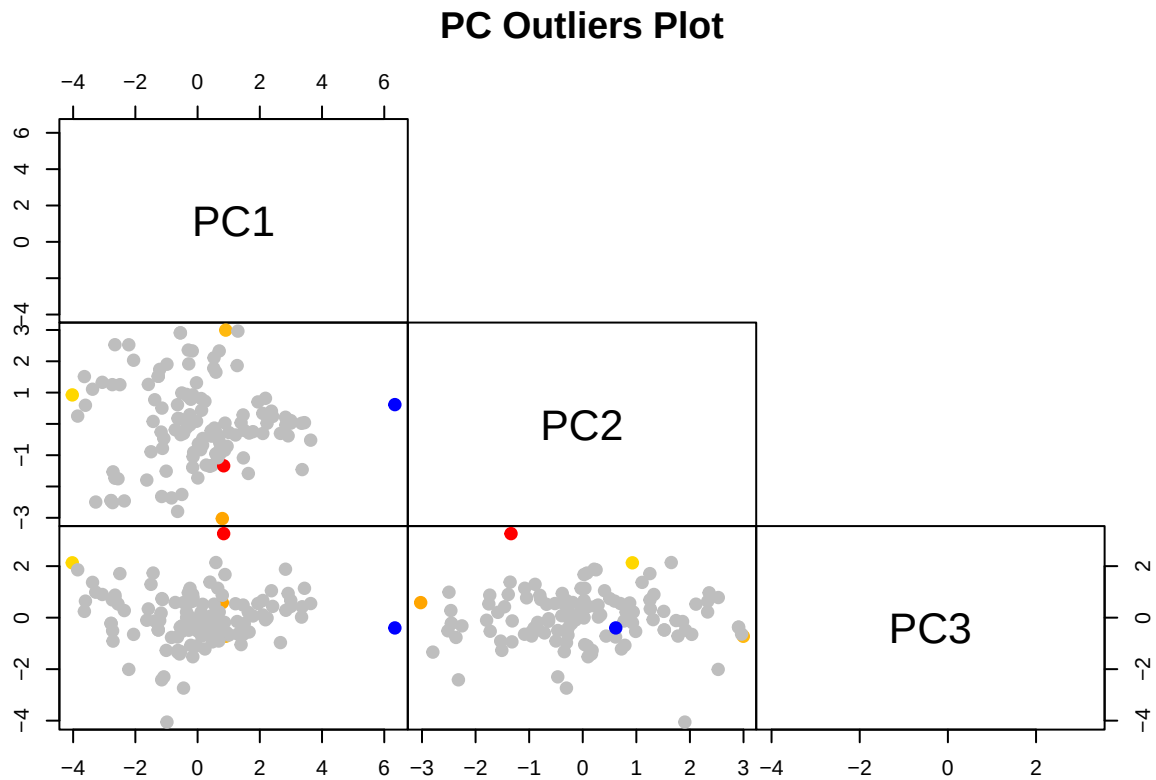


```
fviz_pca_biplot(trainprompt, axes=c(2,3), repel=TRUE, title='Biplot PC2 vs PC3', xlab="PC2 30%", ylab="PC3 16.9%")
```

Biplot PC2 vs PC3



```
scores = trainpca3$scores
outl = c(which.max(scores[,1]), which.min(scores[,1]),
         which.max(scores[,2]), which.min(scores[,2]),
         which.max(scores[,3]))
colors= rep("grey", nrow(mytrainingdatanew))
colors[outl] = c("blue", "gold", "darkgoldenrod1", "orange", "red")
pairs(trainpca3$scores, pch=19, col=colors, upper.panel=NULL, gap=0, cex=1.2, main='PC Outliers Plot')
```



Discussion:

The biplots give a clear visualization of both nutritional variables (loadings) and the observation profiles (scores).

Looking at the variable arrows, the ones pointing in the same direction represent a strong positive correlation. Conversely, arrows in opposite directions have a strong negative correlation and contrast. Furthermore, if an arrow is parallel to an axis this implies that the variable is almost totally explained by that principal component.

Analyzing the PC1 vs PC2 biplot, we can see a continuous gradient and an example of the almost parallel behavior. Indeed “fat_total” is almost aligned with the horizontal PC1 axis, showing how the first component largely captures fat density, meanwhile “sugars” is almost aligned with the y-axis. As a consequence, the products on the extreme right are the most calorific and fat dense, meanwhile the one on the top-left are the most moisturized.

The PC1 vs PC3 biplot allows us to differentiate the products based on secondary traits. We can see that even with a similar calories range product can vary in their nutritional profiles. It contrast highly fibrous product against heavily sweetened ones.

The PC2 vs PC3 biplot is able to examine the data independently of the primary caloric and fat density gradient. As a matter of fact, the arrows of “fat_total” and “energy_kcal” are short because their variance was captured by PC1. We can see how this plot maps the hydration/sodium contrast against sugar/fibres content.

The pairwise scatterplot of the scores shows us the extreme observations in the PCA space, since these points lie far from the center of the PCA subspace they have a large score distance. In general we can divide outliers into three categories: orthogonal outliers (only high orthogonal distance), bad leverage points (high

orthogonal distance and high score) and good leverage points (high score and regular orthogonal distance). It is easy to notice how the extreme points in our plot are leverage points because of their outlying score distances. However, because the score plot does not display orthogonal distances we can't classify them as good or bad leverage points. We will be able to classify them using the Robust PCA Outlier Map in the following question.

Question 5

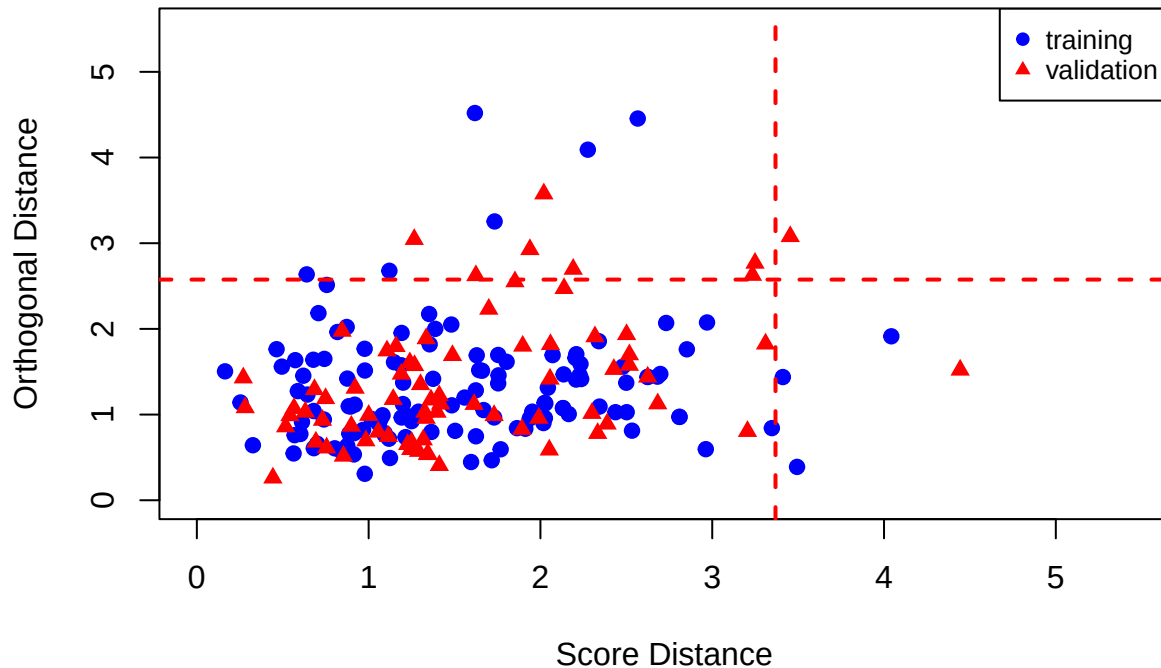
We now have to perform a robust PCA.

Part a

We plot an outlier map with observations from both the training and validation set. Are there any outliers that affect the validation set?

```
trainpcarobust=PcaHubert(mytrainingdatanew, k=3, crit.pca.distances = 0.99, scale=mad)
validationpredict=predict(trainpcarobust, newdata=myvalidationdatanew)
sdtrain=trainpcarobust@sd
odtrain=trainpcarobust@od
valscores=predict(trainpcarobust, newdata=myvalidationdatanew)
eigenvals=trainpcarobust@eigenvalues[1:3]
sdval=sqrt(rowSums(sweep(valscores^2, 2, eigenvals, "/")))
valscaled=scale(myvalidationdatanew, center=trainpcarobust@center, scale=trainpcarobust@scale)
valreconstructed=valscores %*% t(trainpcarobust@loadings)
odval=sqrt(rowSums((valscaled - valreconstructed)^2))
cutoffsd=trainpcarobust@cutoff.sd
cutoffod=trainpcarobust@cutoff.od
maxsd=max(c(sdtrain, sdval, cutoffsd, na.rm=TRUE)) + 1
maxod=max(c(odtrain, odval, cutoffod, na.rm=TRUE)) + 1
plot(sdtrain, odtrain, pch=19, col='blue', xlim=c(0, maxsd), ylim=c(0, maxod), xlab="Score Distance", ylab="Orthogonal Distance", las=1)
points(sdval, odval, pch=17, col='red')
abline(v=cutoffsd, col='red', lty=2, lwd=2)
abline(h=cutoffod, col='red', lty=2, lwd=2)
legend('topright', legend=c('training', 'validation'), col=c('blue', 'red'), pch=c(19,17), cex=0.8)
```

Outlier Map with Robust PCA



Discussion:

We are plotting the score distance (x-axis) against the orthogonal distance (y-axis) noting that the red lines represent the cutoff values, the red triangles represent validation data and the blue dots represent the training one.

If we analyze the four quadrants we can classify the observations in four categories. The regular observations in the bottom-left quadrant are the one which perfectly fit the robust PCA subspace (the majority of training and validation points are here). The good leverage points in the bottom-right quadrant have extreme score distances but fall below the orthogonal cutoff, so they help stabilize the components (only a few validation and training points are here). The orthogonal outliers in the top-left quadrant don't fit the 3D subspace of the robust PCA (several training and validation data are here). The bad leverage points are in the top-right quadrant have both a high score and a high orthogonal distance (only one validation point here).

We can conclude that some outliers affect the classical PCA analysis. Indeed the orthogonal outliers represent a problem, this happens because classical PCA try to minimize the sum of all orthogonal distances, in doing so it would move the principal components towards these points. As a consequence it would fail in capturing the nutritional variance of the majority of the sweets. Furthermore, if the bad leverage point would have been part of the training data it would have distorted the model even more.

Part b

```
outliersindex=which(odtrain>cutoffod)
print(mytrainingdatanew[outliersindex, ])
```

```
##      energy_kcal water carbohydrates_available sugars transfibrescox fat_total
## 41          465  10.5                59    55      1.047170      22
## 64          465   1.0                60    21      3.047369      18
## 65          515   5.0                60    36     -1.867192      27
## 80          475   3.0                64    25      1.907760      19
## 96          415  12.0                48    13      3.432949      18
## 122         480   5.0                59    34      1.907760      22
##      fatty_acids_saturated  Na
## 41                    19.0 0.02
## 64                    7.0 0.26
## 65                    16.0 0.10
## 80                    7.0 0.69
## 96                    6.0 0.18
## 122                   4.5 0.40
```

```
apply(mytrainingdatanew,2, median)
```

```
##      energy_kcal      water carbohydrates_available
##      490.00000      4.50000      60.00000
##      sugars      transfibrescox      fat_total
##      33.00000      1.04717      24.00000
##      fatty_acids_saturated      Na
##      9.10000      0.10000
```

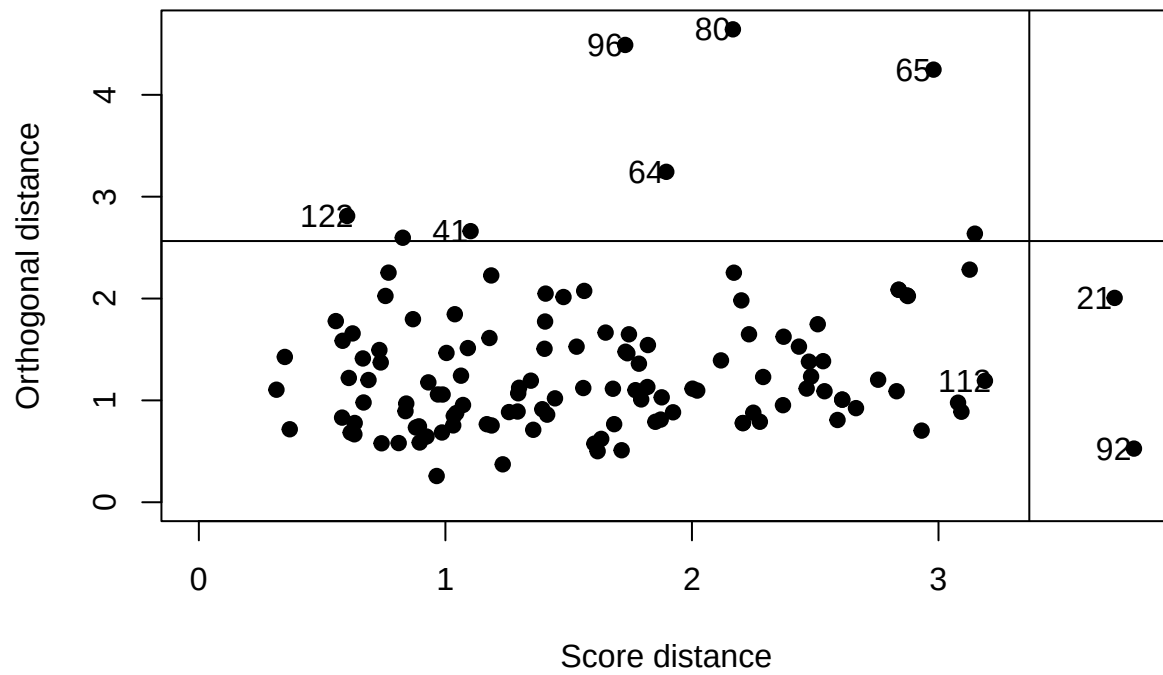
Discussion:

In the previous point we identified possible orthogonal outliers. We have now calculated the outliers (41,64,65,80,96,122), then we compare them with the global medians of the training set allowing us to understand why these observations were isolated by robust PCA. We can see how observation 41 has extremely high “sugars” and “fatty_acids_saturated”. Observation 80 has an extremely high “Na”(saltiness) compared to the median. In observation 96 “water” is extremely high while “sugars” and “carbohydrates_available” are extremely low compared to the median. We can conclude how these observations are flagged because they represent abnormal recipes that don’t follow the general nutritional trend. So their anomaly forces them out of the 3D subspace estimated by the robust model, as a consequence we get high orthogonal distances.

Part c

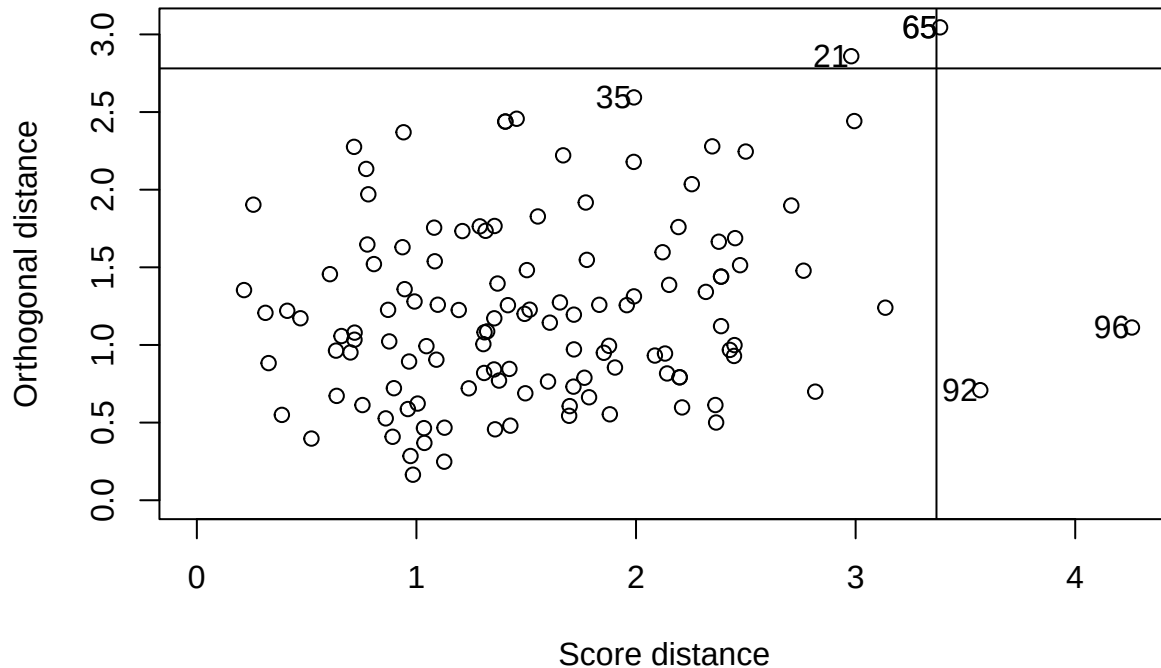
```
trainpcarobust=PcaHubert(mytrainingdatanew, k=3, crit.pca.distances=0.99, scale=mad)
plot(trainpcarobust, pch=19, id.n.od=6, main='Outlier Map with Robust PCA')
```

Outlier Map with Robust PCA



```
plot(trainpca3, main='Outlier Map with Classical PCA')
```

Outlier Map with Classical PCA



Discussion:

Now we compare the outlier maps of the classical PCA and the robust PCA, noting how we can clearly observe the ‘masking effect’ of extreme values on classical methods.

In Question 3, we observed that the classical tolerance ellipse inflates to accommodate outliers. The same mathematical issue can be seen here: in classical map a very small number of observations is flagged as outliers. Because classical PCA uses the standard mean and covariance it is extremely sensitive to extreme values. As a result, the orthogonal distances are minimized and appear normal.

Conversely, because robust PCA uses highly resistant estimators it can’t be distorted by extreme values. This allow us to see the real structure of the data, indeed outliers can’t distort the model. Consequently, the extreme observations are identified as orthogonal outliers and bad leverage points.

We can conclude that the classical PCA yields a distorted model, meanwhile the robust PCA is reliable and capable of spotting real outliers providing a good and accurate representation of the dataset.